CADT
IDT វិទ្យាស្ថានបច្ចេកវិទ្យាឌីជីថល
Institute of Digital Technology

Secure Programming

1 Introduction to Secure Programming


LAY
Leangsros


leangsros.lay@cadt.edu.kh


089909978

About Me

LAY Leangsros

Contact: 089909978 leangsros.lay@gmail.com

1. Licensed Penetration Tester (LPT-Master)
2. Certified Penetration Testing Professional (CPENT)
3. Certified Ethical Hacker (CEH)
4. Comptia Security+
5. Certified Cybersecurity Technician (CCT)
6. Certified Fundamentals of Engineer Exam (IP)
7. Certified Information Technology Passport (FE)

Course Outline

1. Introduction to Software Security Principles
2. The OWASP Top 10
3. Cryptography
4. Injection & Defensive Techniqure
5. Application Security Testing
6. Parameter Logic Bugs and API Security
7. Secure Authentication & Authorization
8. Secure Software Development Lifecycle & DevSecOps
9. Obfuscation & Reverse Engineering
10. Mobile Security

Course Prerequisites

1. Basic knowledge of OS commands and programming language
2. VS Code
3. Burp Suite
4. VMWare Workstation/Kali Pre-built Virtual Machines/Kali WSL
5. Docker

Objectives

- Describe the importance of software security.
- Describe each of the software security principles.
- Apply each of the software security principles.
- Interpret the security and privacy requirements of different systems.
- Analyze and identify any threats and risks for mission-critical requirements.
- Develop secure software design.
- Identify vulnerabilities specific to a platform or language.
- Design code to mitigate common vulnerabilities.
- Identify causes of race conditions
- Implement code review.
- Differentiate between white box, grey box, and black box testing techniques.
- Identify the strengths and weaknesses of static and dynamic analysis tools.

OWASP Top 10

OWASP Top 10	Proactive Controls
•A01:2021- Broken Access Control	•C1: Implement Access Control
•A02:2021- Cryptographic Failures	•C2: Use Cryptography the right way
•A03:2021 - Injection	•C3: Validate, Escape, Sanitize
•A04:2021 - Insecure Design	•C4: Use Secure Architecture Patterns
•A05:2021 - Security Misconfiguration	•C5: Secure By Default Configurations
•A06:2021 - Vulnerable and Outdated Components	•C6: Assess and Update your Components
•A07:2021 - Identification and Authentication Failures	•C7: Implement Digital Identity
•A08:2021 - Software and Data Integrity Failures	•C8: Help the Browser defend its User
•A09:2021 - Security Logging and Monitoring Failures	•C9: Implement Security Logging and Monitoring
•A10:2021 - Server-Side Request Forgery (SSRF)	•C10: Stop Server-Side Request Forgery

OWASP Mobile Top 10 2024

- M1: Improper Credential Usage
- M2: Inadequate Supply Chain Security
- M3: Insecure Authentication/Authorization
- M4: Insufficient Input/Output Validation
- M5: Insecure Communication
- M6: Inadequate Privacy Controls
- M7: Insufficient Binary Protections
- M8: Security Misconfiguration
- M9: Insecure Data Storage
- M10: Insufficient Cryptography

Introduction to Secure Programming

- Secure Programming, also known as **Secure coding** is the practice of writing software that's protected from vulnerabilities.

1.
Introduction to Software Security Principles

Introduction to Cybersecurity

- Cybersecurity refers to the practice of protecting
  - **computer systems**, **networks**, **programs**, and **data** from digital attacks, unauthorized access, damage, or theft.
- Cybersecurity measures involve both technical and non-technical aspects.

"It takes 20 years to build a reputation and few minutes of cyber-incident to ruin it." –
Stephane Nappo

Introduction to Software Security

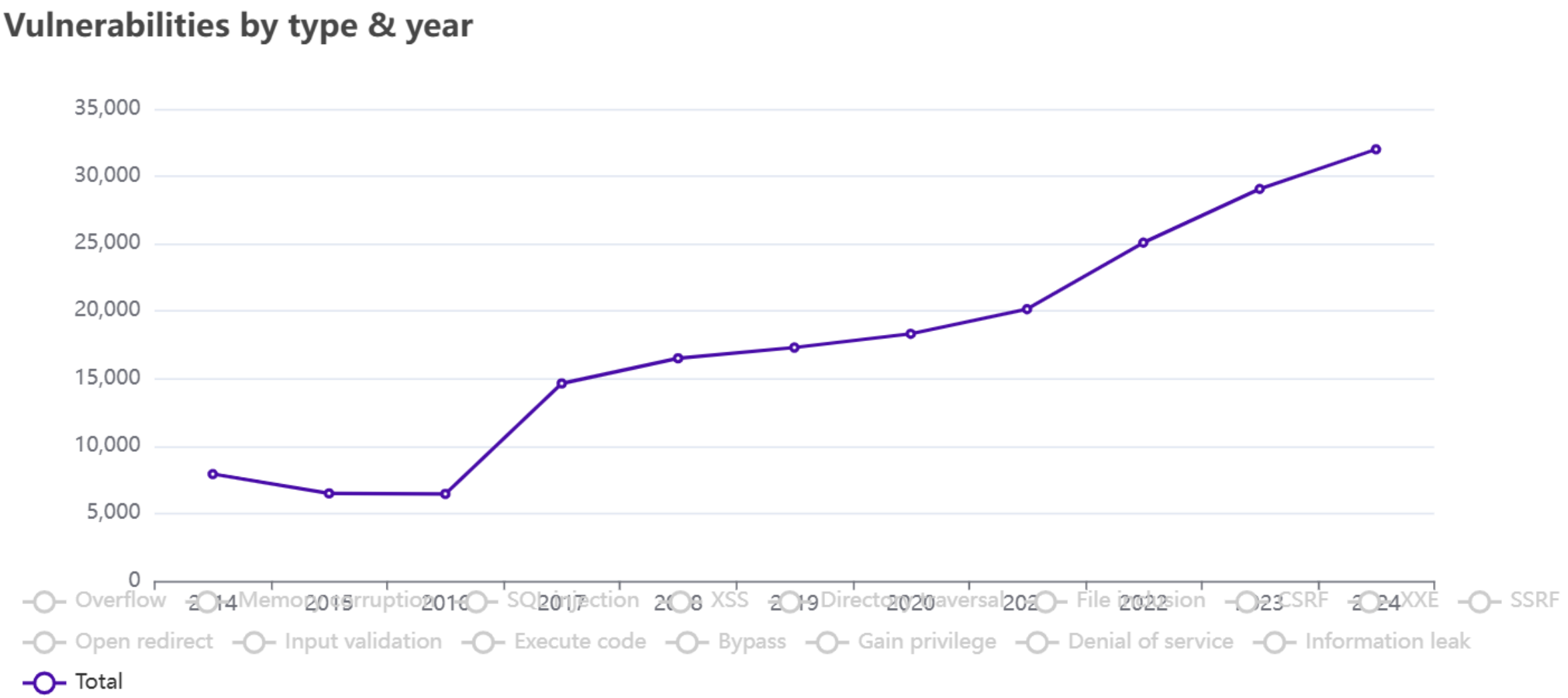
- Software bugs are errors, mistakes, or oversights in programs that result in unexpected and typically undesirable behavior
- Secure software is achieved by anticipating the many attack scenarios and protecting against hypothetical attacks that might violate confidentiality, integrity, or availability
  - Confidentiality: how sensitive is the data and how much of it might be disclosed? 6
  - Integrity: how valuable is the data, how much could be tampered with, and how likely would that be detected?
  - Availability: how long and to what degree would the system be unavailable?

Introduction to Software Security

- Software bugs are errors, mistakes, or oversights in programs that result in unexpected and typically undesirable behavior
- Secure software is achieved by anticipating the many attack scenarios and protecting against hypothetical attacks that might violate **confidentiality**, **integrity**, or **availability**
  - **Confidentiality**: how sensitive is the data and how much of it might be disclosed?
  - **Integrity**: how valuable is the data, how much could be tampered with, and how likely would that be detected?
  - **Availability**: how long and to what degree would the system be unavailable?

Vulnerabilities

- Number of CVEs by year



Vulnerabilities

- SQL injection
- Authentication
- Path traversal
- Command injection
- Business logic vulnerabilities
- Information disclosure
- Access control
- File upload vulnerabilities
- Race conditions
- Server-side request forgery (SSRF)
- XXE injection
- NoSQL injection

Threat ✖ Vulnerability = Risk

SQL injection (SQLi)

- A successful SQL injection attack can result in unauthorized access to sensitive data, such as:
  - Stolen credentials, (Passwords), Credit card details, Personal user information.
  - Data alteration, Data deletion
- Common SQL injections
  - **In-band SQLi (Classic SQLi)**
    - **Error-based SQLi**
    - **Union-based SQLi**
      - SELECT a, b FROM table1 UNION SELECT c, d FROM table2
  - **Inferential SQLi (Blind SQLi)**
    - **Boolean-based (content-based) Blind SQLi**
    - **Time-based Blind SQLi**
  - **Out-of-band SQLi**

SQL injection (SQLi)

Prevention

- Option 1: Use of Prepared Statements (with Parameterized Queries)
- Option 2: Use of Properly Constructed Stored Procedures
- Option 3: Allow-list Input Validation
- Option 4: STRONGLY DISCOURAGED: Escaping All User Supplied Input

Secure Coding Principles

Four Pillars of Software Security

- Secure by Design
- Secure by Default
- Secure by Implementation
- Secure by Communication

Secure Coding Principles

Four Pillars of Software Security

- Secure by Design
  - products are built to protect against malicious cyber actors so they cannot access devices, data, or connected infrastructure
- Secure by Default
  - Security is included in the base product at no additional cost,
  - products are configured to protect against the most prevalent threats and vulnerabilities
- Secure by Implementation
  - Thoroughly execute security measures within your code to leave no room for vulnerabilities.
- Secure by Communication
  - Foster secure channels for data exchange, safeguarding information while in transit.

Secure Coding Principles

- Secure By Design
  - Establish Trust Boundaries
  - Don't Reinvent the Wheel
  - Economy of Mechanism
  - Trust Reluctance
  - Open Design
  - Minimize the Attack Surface
  - Secure the Weakest Lin**Security**
    k
- Secure By Default
  - Use Least Privilege
  - Use Default Deny
  - Fail Securely
- Secure By Communication
  - Secure Trust Relationships
- Secure by Implementation
  - Psychological Acceptability
  - Least Common Mechanism
  - Validate Inputs
  - Secure Data at Rest
  - Prevent Bypass Attacks
  - Audit and Verify
  - Defense in Depth

Secure Coding Principles

Secure By Design

Trust Boundaries

- *Trust boundaries* are uncontrolled input from an external source – like HTTP requests, file uploads, or network sockets – are taken into a controlled environment, like a web-server.
  - *Bypass Protection Mechanism*
- All input is evil
- **CWE-501: Trust Boundary Violation**
  - A trust boundary violation occurs when a value is passed from a less trusted context to a more trusted context.
- Validate data before putting into session variable. Use regular expression or whitelist input validation at server side.

Secure Coding Principles

Secure By Design

Trust Boundaries

```
String username = request.getParameter("username");
String password = request.getParameter("password");
HttpSession session = request.getSession(true);
session.setAttribute("username", username);
session.setAttribute("validated", false);
if (!this.credentialsAreValid(username, password)) {
    request.setAttribute(“msg", "Incorrect credentials");
    response.sendRedirect("/login");
    return;
}
session.setAttribute("validated", true);
response.sendRedirect("/home");
```

```
String username = request.getParameter("username");
String password = request.getParameter("password");
if (!this.credentialsAreValid(username, password)) {
    request.setAttribute("msg", "Incorrect credentials");
    response.sendRedirect("/login");
    return;
}

HttpSession session = request.getSession(true);
session.setAttribute("username", username);
response.sendRedirect("/home");
```

Secure Coding Principles

Secure By Design

Trust Boundaries

- Never trust Data, validate everything
- Implement black-lists / white-lists
- Whitelist — simple example

```
private static final String[] ALLOWED_FILE_EXTENSIONS = {".gif", ".jpeg", ".png"};
static boolean isValidFileExtension(String fileName) {
    for (String ext : ALLOWED_FILE_EXTENSIONS) {
        if (fileName.endsWith(ext)) {
        return true;
        }
    }
    return false;
}
```

Secure Coding Principles

Secure By Design

Trust Boundaries

- Regular expression for dates in a YYYY-MM-DD format, and valid date

```
private static final Pattern DATE_PATTERN =
Pattern.compile("(\\d{4})-(\\d{2})-(\\d{2})");
static boolean isValidDate(String s) {
    Matcher m = DATE_PATTERN.matcher(s);
    if (m.matches() ) return true;
     return false;
}
//2024-04-31
```

Secure Coding Principles

Secure By Design

Trust Boundaries

- Regular expression for dates in a YYYY-MM-DD format, and valid date

```
private static final Pattern DATE_PATTERN = Pattern.compile("(\\d{4})-(\\d{2})-
(\\d{2})");
static boolean isValidDate(String s) {
    Matcher m = DATE_PATTERN.matcher(s);
    if (! m.matches() ) return false;

    int year = Integer.parseInt(m.group(1));
    int month = Integer.parseInt(m.group(2));
    int day = Integer.parseInt(m.group(3));

    return month >= 1 &&
    month <= 12 &&
    day >= 1 &&
    day <= daysInMonth(month, year);
}
```

Secure Coding Principles

Secure By Design

Trust Boundaries

```
System.out.println(Integer.MIN_VALUE); // -2147483648
System.out.println(Integer.MAX_VALUE+1); // -2147483648
int v = 1073741824; // 2^30
System.out.println(2 * v); // -2147483648
System.out.println(4 * v); // 0
System.out.println(Integer.MIN_VALUE + Integer.MIN_VALUE); // 0
```

Secure Coding Principles

Secure By Design

Trust Boundaries

The value “to pay” will be 0 if price == 4 and quantity == 1073741824

```
import java.util.Scanner;
class Payment {
    public static void main(String[] args) {
        try(Scanner in = new Scanner(System.in)) {
            int price = in.nextInt();
            int quantity = in.nextInt();
            if (price <= 0 || quantity <= 0) {
                System.out.println("Invalid request!");
            } else {
                int toPay = price * quantity;
                System.out.printf("OK! The price to pay is %d\n", toPay);
            }
        }
    }
}
```

```
4
1073741824
OK! The price to pay is 0
```

Secure Coding Principles

Secure By Design

Don't Reinvent The Wheels

- When there is good security code available, don't try to create your own
  - –Security is difficult to do
  - –Mature frameworks can provide features you wouldn't have time to develop
- There is a lot of code available
  - –Some of it has been written by talented people
  - –Some of it has been thoroughly vetted and tested
  - –Some of it is known to be secure or more secure than untested code
  - –Check the Common Vulnerability Database for current vulnerabilities

Secure Coding Principles

Secure By Design

Don't Reinvent The Wheels

- Cryptography is difficult to implement well
  - If you are creating your own cryptography methods, do you have the numerical expertise to know it is secure?
  - If you are implementing existing algorithms, can you get it perfectly?
  - Anything less than perfect will be vulnerable
  - The amount of time you invest will be far larger than the time required to use an existing library, which is likely free
  - If you intend to seek any compliance certifications, they will require the use of a certified cryptography library

Secure Coding Principles

Secure By Design

Don't Reinvent The Wheels

- Consider using an Application Framework, Many have built-in support for:
  - An extensive authentication and session model
  - Input validation
  - SQL Injection avoidance
  - XSS avoidance
  - Path traversal avoidance
  - Cross-site Request Forgery protection

Secure Coding Principles

Secure By Design

Economy of Mechanism

- Principle: Security mechanisms should be as simple as possible
- The **KISS** adage, "Keep It Simple Stupid," applies to security
  - Complicated is the enemy of security
    - High complexity leads to more defects
    - Complicated code is more difficult to test and patch
    - Adding security means more code
  - Simple security constructs
    - Are more likely to be defect-free
    - Require less development and test time
  - Don't implement unnecessary security constructs

Secure Coding Principles

Secure By Design

Economy of Mechanism

- The IPSEC specification has over a dozen RFC's and has hundreds of pages
- Early implementations had many serious vulnerabilities
- There are numerous failures of security features due to the complexity of setting and processing the various cookies

Secure Coding Principles

Secure By Design

Trust Reluctance

- Principle: Privileges should not be granted based on a single condition
- Prevent the breach of one component leading to the breach of others
- Trust Partitioning, Trust Distribution, Separation of Privilege
- A way to minimize the effect of security breach
- To increase the flexibility of the application without compromising security
- To operate securely in unsecured environment

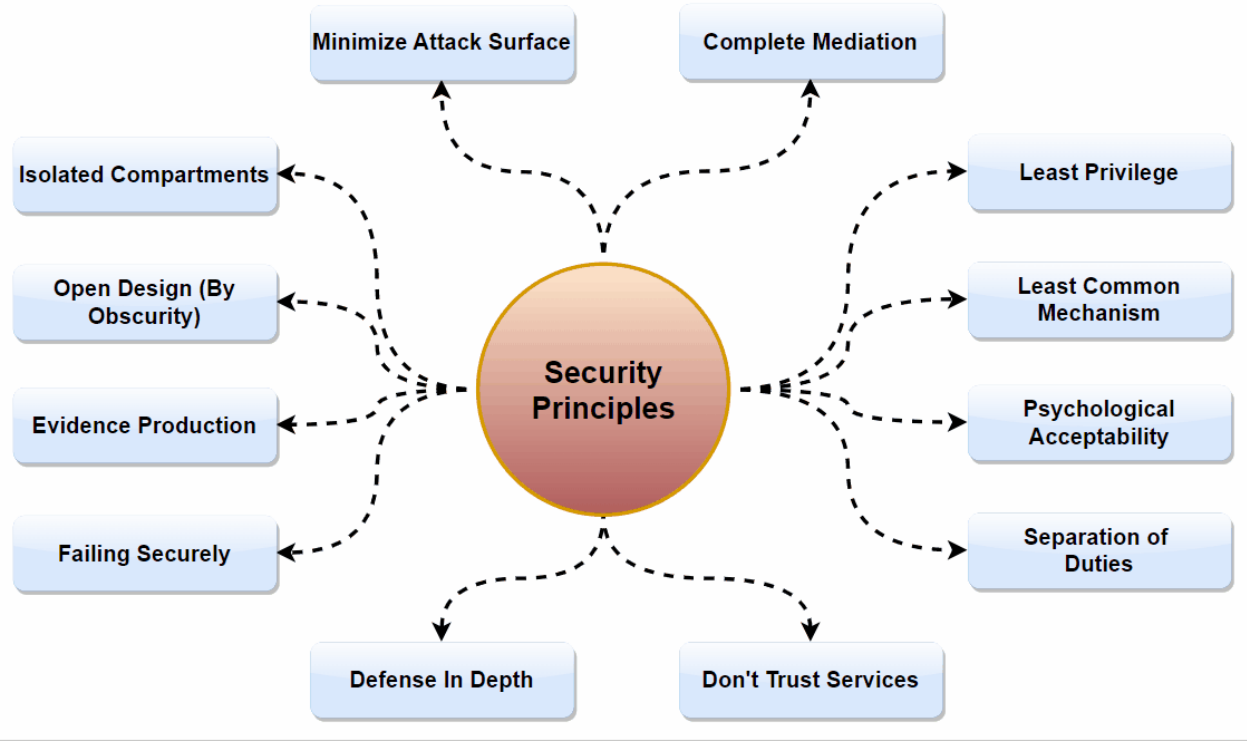
Secure Coding Principles

Secure By Design

Trust Reluctance

Tips:

- Promote privacy
- Compartmentalize
- Defend in Depth
- Use Community resource-no security by obscurity
- Least Privilege,
    - deny them access to features, privileges, and controls they won't utilize.



Secure Coding Principles

Secure By Design

Trust Reluctance

Tips:

- Trust must be created, not assumed
  - If a user is authenticated, they should also have to have authorization to access a resource
- Minimize the set of components to be trusted
  - –Personnel, systems, operations
  - –Fewer trusted components means fewer secure systems to insure safety

Secure Coding Principles

Secure By Design

Open Design

- Principle: The security of a component or system should not depend on the secrecy of the design or implementation
- Crypto-systems should remain secure even when the attacker knows all the internal details
- Also known as avoiding "**Security by Obscurity**"
- It is highly unlikely that any algorithm or method can be kept secret
  - Many people know
  - Attackers can guess and probe the application
  - Your own documentation may reveal the secrets

Secure Coding Principles

Secure By Design

Open Design

- You should not depend on obscurity for protection
- Security Through Obscurity (STO) uses secrecy as a primary method for protecting systems but does not provide direct protective measures against attacks and should not be the sole security strategy.
- Assume all potential attackers know everything about the design
- Making it more difficult, is never the wrong thing to do

Secure Coding Principles

Secure By Design

Open Design

- Design security features as though only the keys are private
  - The point of open design is that secrets usually are not secret
  - If your software has proprietary algorithms, processes and procedures, assume they are public when you design
  - Without proprietary algorithms, you need to be especially careful with keys
- Areas where data is commonly assumed to be safe when it is not
  - Registry keys
  - Hard-coded passwords or encryption keys
  - Data or code in HTML pages
  - Anything stored on a client host
  - Anything sent over an unencrypted communication channel
- Keys are safe only if you make them so
  - –Inside of your Trust Boundaries
  - –Security features designed to protect them even if everyone knows how they work

Secure Coding Principles

Secure By Design

Minimize the Attack Surface

- Principle: The smaller the attack surface, the safer the application
- The risk to an application is:
  - Size of the Attack Surface x The Probability of a Vulnerability
  - The probability of a vulnerability is not going to be zero
- Attack surface reduction (ASR) requires an acceptance of the idea that code is never completely secure
  - Design or implementation failures
  - Poor configurations
  - Failure to install patches or upgrades
  - New attack types
  - Mistakes

Secure Coding Principles

Secure By Design

Minimize the Attack Surface

Tips

- Reduce the amount of running code
  - only necessary code should be running
  - modularizing and isolating software components
  - Consider a distributed architecture
  - Limit access to code when not universally required
- Minimize open ports and services
  - Only allow services to run that are necessary
  - Close all unnecessary ports
- Reduce the number of entry points
  - Remove untrusted components that can access the system
- Attack Points
  - UI Forms and fields
  - HTTP headers
  - Cookies
  - API's and API functions
  - Login/authentication's
  - Interfaces with other systems
  - Database interfaces
  - Admin interfaces

Secure Coding Principles

Secure By Design

Secure the Weakest Link

- Principle: Attackers will attack the weakest security point in the application
- Factor in the four sources of threats
  - Social: People
  - Operational: Processes (and People)
  - Technological: The application and network
  - Environmental: Facilities

Secure Coding Principles

Secure By Design

Secure the Weakest Link

- Default settings on the application or servers
- Backdoors
- Test servers
- A wireless network
  - printers can be attacked in many ways like Denial of service, protection bypass, print job manipulation, Information disclosure, Privilege Escalation etc.
- That old backup server no one uses
- People who are negligent or ill-informed
- Open ports in the firewall or server
- Networked printers, in general, are poorly secured.

Secure Coding Principles

Secure By Default

Use Least Privilege

- Principle: A subject should only be granted only the privileges needed for an operation
- Least Privilege is a concept that means that at any given application state, the user will operate at the lowest level of access rights possible
- A program should be given only those privileges it needs in order to satisfy its requirements—no more, no less
- Consequences:
  - In case of a breach, damage is limited
  - Cause of breach can be analyzed and fixed
  - Greatly improved overall security
- Laziness/In-a-hurry are the enemies of this principle

Secure Coding Principles

Secure By Default

Use Least Privilege

- The "Least Privilege" concept will not stop application security weaknesses, like code injections, for example. However, it will make it much harder for an attacker to further exploit those weaknesses
- Application users should also have as little privileges as possible, while still allowing them to perform their business processes. To do this effectively, Administrators should implement role-based access controls.

Secure Coding Principles

Secure By Default

Use Default Deny

- Principle: Anything not specifically allowed must be denied
  - Default configurations should be the most secure setting
  - Users often don't change default configurations, or even look at them
- Fail-Safe Defaults
- The access control system should default to no access
- In the event of an operation failure, do nothing
  - And restore the system to the state prior to attempting the operation
- Trust Boundaries can be used to motivate this effort

Secure Coding Principles

Secure By Default

Fail Securely

- Principle: Handle all failures securely and return the system to a proper state
- Failure:
  - Error messages can disclose information valuable to an attacker
  - Failure can lead to an unhandled state, which can lead to denial of service
  - Unhandled failures can lead to malicious behavior being unnoticed
- Consequences:
  - All error conditions are handled and logged
  - There are no verbose error messages
  - The failure of a component doesn't result in systemic failure
  - A failure does not result in a secure data breach

Secure By Default

Fail Securely

- When a function or transaction fails, you should leave the system in a secure state

```
// Example 1 (bad code)
isAdmin = true;
try{
    codeThatMayFail();
    isAdmin =
isUserInRole("Administrator");
}catch(Exception e) {
    log.write(e.toString());
}
```

```
// Example 1 (better code)
isAdmin = false;
try{
    codeThatMayFail();
    isAdmin =
isUserInRole("Administrator");
}catch(Exception e) {
    log.write(e.toString());
}
```

- If codeThatMayFail fails, remaining statements within try block will not execute
- An attacker could force an error in example 1 to get admin access

Secure Coding Principles

Secure By Default

Fail Securely

Tips

- Errors are like accidents, you don't expect them, but they happen
- Any code that can throw an exception should be in a Try Block
- Handle all possible exceptions
- Use Finally Blocks: leaving files open or exceptions defined after use creates resource leaks and possible system failure
- Short specific Try Blocks give you more control over the error state

Secure Coding Principles

Secure By Communication

Secure Trust Relationships

- Principle: Trust in outside or Third-Party entities must be established and maintained
  - –Nothing outside of your Trust Boundaries is safe
  - –You can't trust others to be secure
  - –Even the most secure communication has weaknesses
- Most systems of any value will have to do one or more of these
  - –Exchange data with a client system
  - –Exchange data with a Third-Party system

Secure Coding Principles

Secure By Communication

Secure Trust Relationships

Example:

You allow remote access for system management

- Authentication is critically important; consider two factor authentication
  - Using SMS codes
  - Using hardware authenticators: disconnected or connected tokens
  - Biometrics
  - Use combination of what the user knows, has or inherits
- All communications tunnelled through SSH

Secure Coding Principles

Secure By Communication

Secure Trust Relationships

Tips:

- Authentication
  - Use two-factor authentication when required
  - Encrypt all client-side session data; cookies, session tokens
  - Encrypt all authentication credentials in-transit
  - Do not leave authentication information in client memory unless it is encrypted
  - Do not put hard-coded passwords, encryption keys or vital information on the client host
- Communications must be encrypted to defend confidentiality
  - HTTPS uses SSL/TLS for endpoint authentication and encryption
  - Configure SSL/TLS to exclude weak cryptography
  - Use SSH to create an encrypted tunnel for non-HTTP traffic

Secure Coding Principles

Secure By Implementation

Psychological Acceptability

- Principle: Security mechanisms should not make the resource more difficult to access than if the security mechanism were not present
- Security and user-friendliness are often contradictory
- Security mechanisms should seek to be as unobtrusive as possible while still meeting security goals

Secure Coding Principles

Secure By Implementation

Psychological Acceptability

- If the configuration for securing a security mechanism is difficult or confusing, users may choose to not enable it
  - –In the end, that is not good for anyone
- Your company mandates two-factor login for google mail
  - –Is having to get an authentication code from your cell phone when you login psychologically acceptable?
  - –If you only have to do so every 30 days for any given host?
- A college campus decides that usernames should be less "guessable" and replaces them with 8 random characters
  - –Psychologically acceptable?
- A system requires that all personal files be passworded, so each access requires a password to be typed
  - –Psychologically acceptable?

Secure Coding Principles

Secure By Implementation

Psychological Acceptability

- Security is going to reduce convenience
- Security mechanisms that are unusually complex or difficult to use will be evaded or create complaints
- Security mechanisms can often be disabled

Secure Coding Principles

Secure By Implementation

Least Common Mechanism

- Principle: Mechanisms to access resources should not be shared between privilege levels
- Minimize the amount of mechanism common to more than one user and depended on by all users.
- For example, the use of the same function to retrieve the bonus amount of an exempt employee and a non-exempt employee will not be allowed.

Secure Coding Principles

Secure By Implementation

Validate Inputs

- Principle: All inputs should be treated as untrustworthy
- Undefined states are problematical for security because there is no sure way to predict how the application will respond
- There are many vulnerabilities that are created or worsened by using unvalidated inputs in code constructs
  - All of the injection vulnerabilities
    - SQL Injection, XSS, HTTP Response Splitting, Command Injection..
  - Open Redirect
  - Buffer Overflow, Uncontrolled Format String, …
  - Parameter Tampering
- if you allow inputs that are incorrect for any reason into your system, you risk its integrity

Secure Coding Principles

Secure By Implementation

Validate Inputs

- • Always test the big three:
  - –Correct type (null/non-null, text/integer/float, scalar/array/object, …)
  - –Proper length (minimum-to-maximum)
  - –Acceptable content
- Use Whitelisting if possible
- All validation must be done on the server
- Validation performed on the client for user-friendliness must be repeated
- Use regular expressions to test; they are the best descriptions
- Remember that Trust Boundaries are not smooth
  - –An application needs to test all input, regardless of the origin
  - –Input from supposedly trusted systems is subject to errors, corruption and manipulation

Secure Coding Principles

Secure By Implementation

Secure Data at Rest

- Principle: Data at rest must be protected to meet security requirements
- Data stored as part of an application has needs for:
  - –Security, which determines if access granted or not granted for a data set
  - –Privacy, determines what the access means to a given user; what form is the data in what operations are allowed
- An application has information that can be divided into give different types of access:
  - admin, agent, support, user, guest

Secure Coding Principles

Secure By Implementation

Secure Data at Rest

There are three kinds of data

1. Data that is private to everyone except the owner
2. Data that is private to certain groups
3. Data that is not private

Secure Coding Principles

Secure By Implementation

Secure Data at Rest

Data that is private to everyone except the owner

- User passwords
- Cookie contents
- Encryption keys
- Critical data like PCN, SSN or as defined by the requirements
- This data should be encrypted at rest
  - This data should be masked if involved in any operation where it might be seen by anyone other than the owner (and possibly even then)
  - On a client system, this data must not be allowed to persist in an unencrypted form

Secure Coding Principles

Secure By Implementation

Prevent Bypass Attacks

- Principle: Attacks that bypass authentication or authorization gates are among the most dangerous
- In a bypass attack, an attacker:
  - –Is able to act as an authenticated user without providing valid credentials
  - –Is able to access resources by evading the authorization checks
- Many vulnerabilities can result in a bypass, but the focus here is on the authentication and authorization system

Secure Coding Principles

Secure By Implementation

Prevent Bypass Attacks

- Authentication bypass can occur by:
  - –Forceful browsing to parts of the application that fail to authenticate
  - –Stealing or forging session tokens
  - –Stealing/guessing credentials
  - –Masquerading
- • Authorization bypass can occur by:
  - –An authentication bypass
  - –Forceful browsing to parts of the application that fail check privileges
  - –Forging credentials for the authorization system

Secure Coding Principles

Secure By Implementation

Prevent Bypass Attacks

- Credentials are almost always stolen from the client or the exchange between the client and server
- –Cookies can be co-opted by attacks like Cross-site Request Forgery
- –Cookies can be stolen or forged if the attacker has access to a system on which a valid user has authenticated
- –Session or authorization data stored in the URL is subject to being cached and then viewed at some later time
- –Poor messaging might allow the attacker to guess credentials
- –A poorly designed forgotten system could allow attackers to breach user accounts
- –Weak passwords are easy to guess
- –Weak cryptography can allow credentials to be stolen

Secure Coding Principles

Secure By Implementation

Audit and Verify

- Principle: A system can't be secure if its operation can't be audited and verified
- –There must be an audit trail for all operations that have a security component
  - Login/Logout/Password change
  - Data read/write/update/delete
  - Administrative operations, user add/delete, change in access controls
  - Resource access
  - Network connect/disconnect
  - Errors/Failures
- –The audit trail must extend to into the past as long as required
  - Contractual obligations to customers, compliance organizations
  - Legal obligations

Secure Coding Principles

Secure By Implementation

Audit and Verify

Logs

- –Should be treated as critical data
- –Must not contain sensitive data
  - Passwords, encryption keys, payment card information
  - It is possible to allow this data if properly masked
- –Should not be kept on a server that allows user access
- –Must be encrypted if moved outside of a Trust Boundary
- –Must be in locked storage if stored on movable media
- Audit reports should be available for easy access in the event of a security
- incident

Secure Coding Principles

Secure By Implementation

Defense in Depth

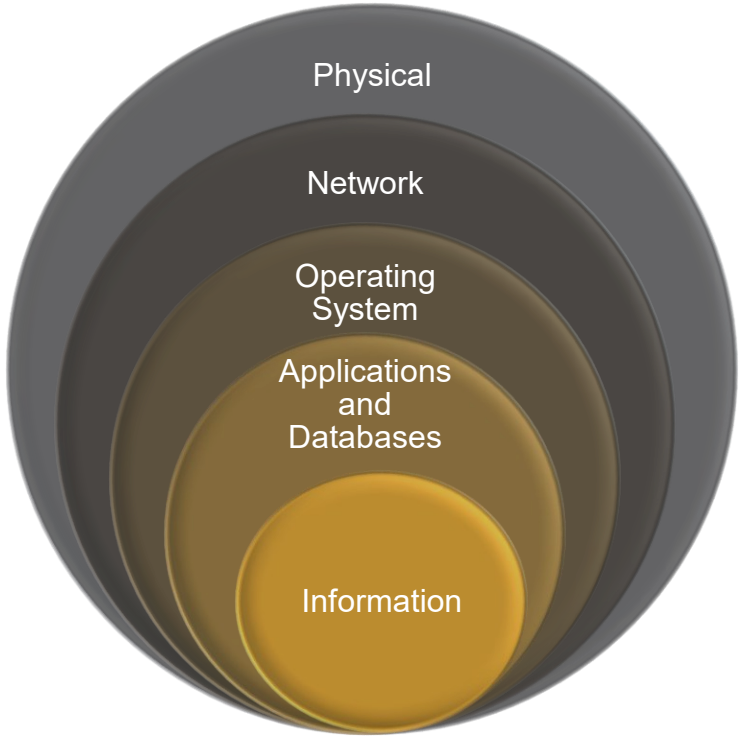
- Principle: Use layered security defenses

Secure Coding Principles

Secure By Implementation

Defense-in-depth

- Physical
  - Physical Access Control System
  - Visitor and Inventory Access Logs
  - Security Guards
  - CCTV Systems and Motion Sensors
  - Guards, Locks, Tracking devices, and Badging systems
- Network
  - Firewall and Configuration
  - Firewall and Router Hardening
  - IDS/IPS
  - Proxy Servers
  - Network segmentation, IPsec, TLS, NAT
  - Routers and Access Control Lists
- Operating System
  - Host IDS/IPS
  - Logging and Log Management
  - OS Hardening, Authentication, Patch management, Antivirus
  - File Integrity Monitoring
- Applications and Databases
  - Database Hardening
  - Encryption and Key Management
  - Application Server Hardening
  - Authentication and Authorization
  - Logging and Log Management
  - Use of SSL/TLS
  - Authentication, Authorization, Auditing (as AAA)
  - Secure Coding Practices
  - Web Application Firewall
- Information
  - Access controls, Encryption, Backup and restore procedures



military strategy : deep defense
information security : layered security